

Back-Chaining Theorem Prover using Unification

Andrew Zehrer

May 1, 2026

[GitHub Link](#)

1 Motivation and Goal

The goal of this project was to build a simplified Prolog-style theorem prover based on first-order logic, using a back-chaining inference engine capable of answering logical queries over a knowledge base. The system should read clauses written in S-expression format, interpret them as facts or rules, and interactively respond to user queries by attempting to prove them.

This project demonstrates how automated reasoning can be constructed from a small set of core ideas: symbolic representation, unification, and recursive goal reduction. The broader motivation is to understand the mechanics behind logic programming languages such as Prolog.

2 Background

In first-order logic, knowledge is typically expressed as **facts** and **rules**. A fact is an atomic statement, while a rule has the form `(implies body head)`. This is read in plain English as "head implies body". To answer a query, a theorem prover attempts to derive the query from the knowledge base. Back-chaining is a goal-driven strategy that works by recursively reducing a goal into subgoals using matching rules. Unification is the key operation that enables this process. The program attempts to make two logical expressions identical by finding a substitution for variables. A correct unification algorithm must also include an occurrence check to prevent infinite substitutions.

3 Methods

3.1 Parsing and Knowledge Base Representation

The system parses input files written in S-expression format. A tokenizer splits the input into parentheses and symbols, and a recursive descent parser constructs nested list representations of expressions.

Each top-level expression is interpreted as either:

- a **fact** (e.g., `male george`), or
- a **rule** (e.g., `(implies (parent ?Y ?X) (child ?X ?Y))`).

The knowledge base is indexed by predicate name into two structures: one for facts and one for rules. This allows efficient lookup during proof search.

3.2 Unification

The unification algorithm recursively matches two expressions under a substitution. The implementation handles variable binding, recursive structure matching, substitution propagation, and occurrence checks to prevent cyclic bindings. Variables are identified syntactically (strings beginning with `?`), and substitutions are stored as mappings from variables to expressions.

3.3 Standardization Apart

To avoid variable name collisions across rule applications, variables in rules are renamed before use. This is implemented via a global counter that ensures each variable instance is unique during proof search. For example: `?X` would become `?X_1`, `?X_2`, etc. each time it is used.

3.4 Back-Chaining Prover

The core procedure, `prove()`, implements a depth-first back-chaining algorithm:

- For atomic goals, it attempts to unify the goal with known facts.
- It also matches the goal with rule heads, and recursively proves the corresponding rule bodies.
- Logical connectives are supported:
 - `and`: sequentially prove subgoals,
 - `or`: try alternative subgoals,
 - `not`: succeed if the subgoal fails (negation as failure),
 - `=`: enforce equality via unification.

The system maintains a list of substitutions (solutions) and propagates them through recursive calls.

3.5 Interactive Query Loop

A simple command-line interface allows users to input queries in S-expression form. For each query, the query is parsed, the prover attempts to find all solutions, and variable bindings are printed, normalized and deduplicated.

4 Implementation Details

One important detail is the handling of substitutions. Before any unification step, expressions are normalized by applying existing substitutions. This ensures consistency across recursive calls. Another key detail is solution normalization. Since multiple derivations may produce equivalent bindings, duplicate results are removed before printing. The interface also includes a `--trace` option, which prints the recursive proof steps with indentation. This is useful for debugging and understanding the search process. Finally, the code separates parsing, unification, and proof search into modular components, making the design easier to reason about and extend.

5 Results

The implemented system successfully supports:

- querying facts
- chaining rules through recursive reasoning
- variable binding and propagation
- logical connectives (and, or, not)
- interactive querying via a command-line interface

Given a properly defined knowledge base in S-expression format, the prover can derive multiple valid solutions and display the corresponding variable bindings. The use of standardization apart ensures correctness across repeated rule applications, and the occurrence check prevents invalid unifications. While the system is not optimized for performance, and may struggle on high-depth recursion, it captures the essential behavior of a Prolog-style interpreter and demonstrates the core principles of logic programming.

6 Conclusion

This project is heavily related to the coursework in CSCE 689, since it combines and applies the topics of unification, first-order logic, and theorem proving. It illustrates how a working theorem prover can be constructed from a unification algorithm and a recursive proof strategy. By combining symbolic parsing, substitution-based reasoning, and back-chaining, the system achieves a functional subset of Prolog. The implementation highlights both the power and the limitations of simple logical inference systems, and provides a foundation for further extensions such as optimization, indexing improvements, or more sophisticated search strategies.